MUX, DMUX, Encoder, Decoder, priority encoder

Digital System Design

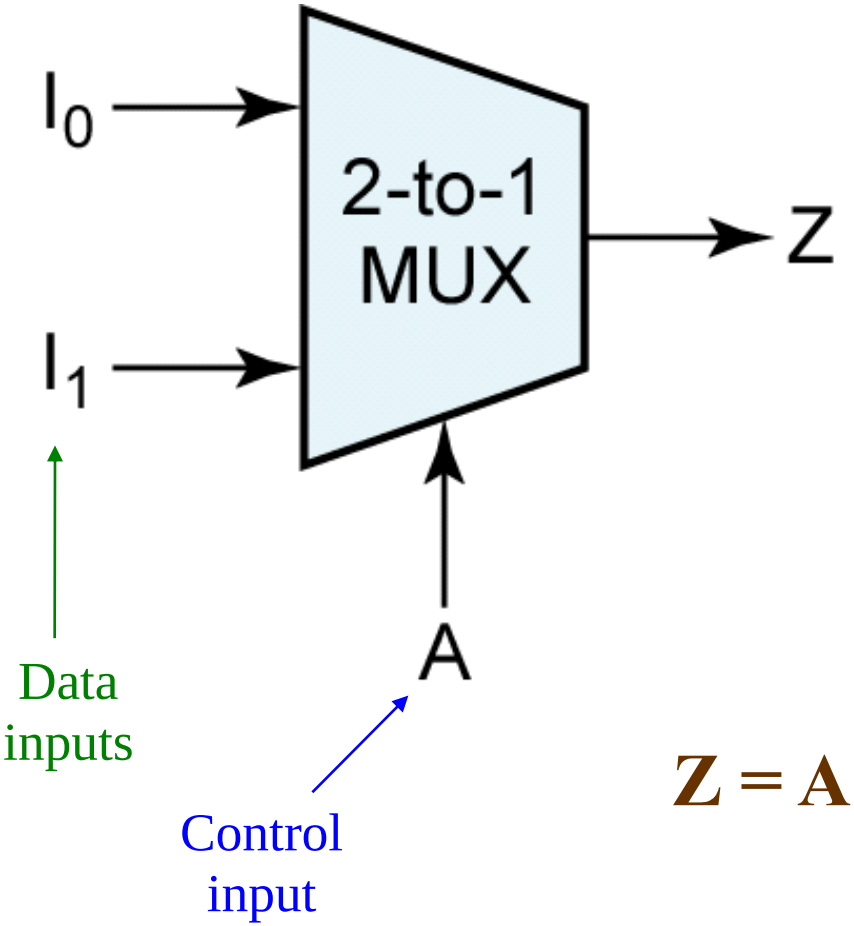
Multiplexers and Demultiplexers, and Encoders and Decoders

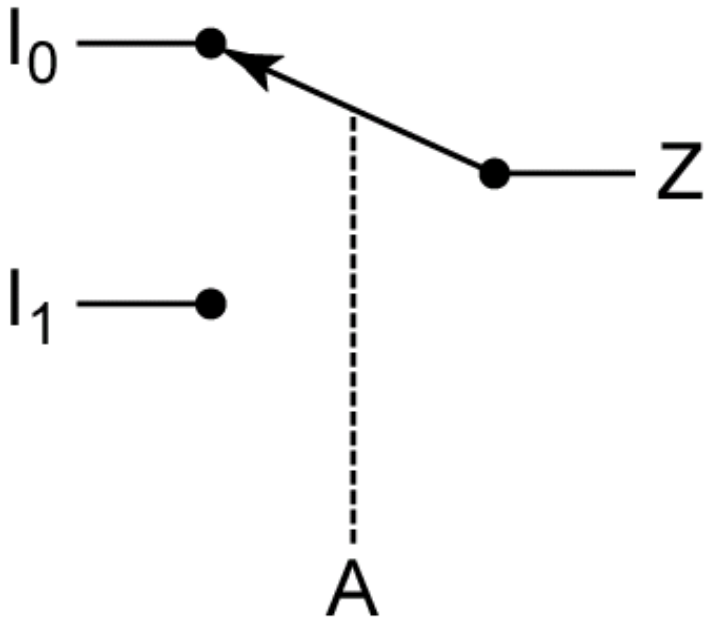
Multiplexers

Multiplexers

- A multiplexer has
  - N control inputs
  - 2^N data inputs
  - 1 output
- A multiplexer routes (or connects) the selected data input to the output.
  - The value of the control inputs determines the data input that is selected.

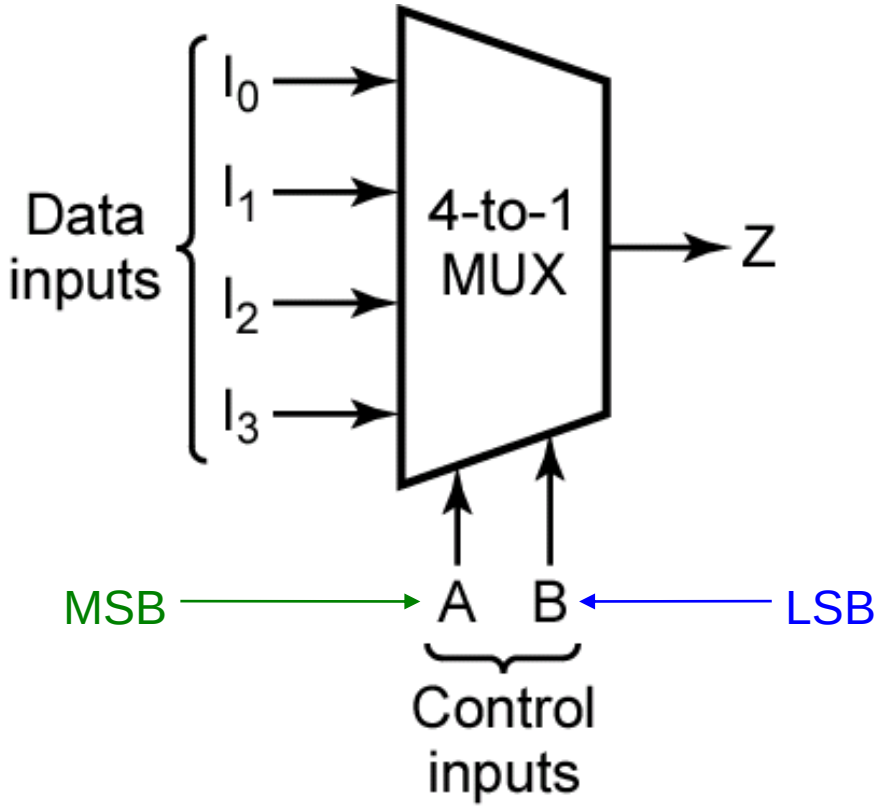
Multiplexers





$$Z = A'.I_0 + A.I_1$$

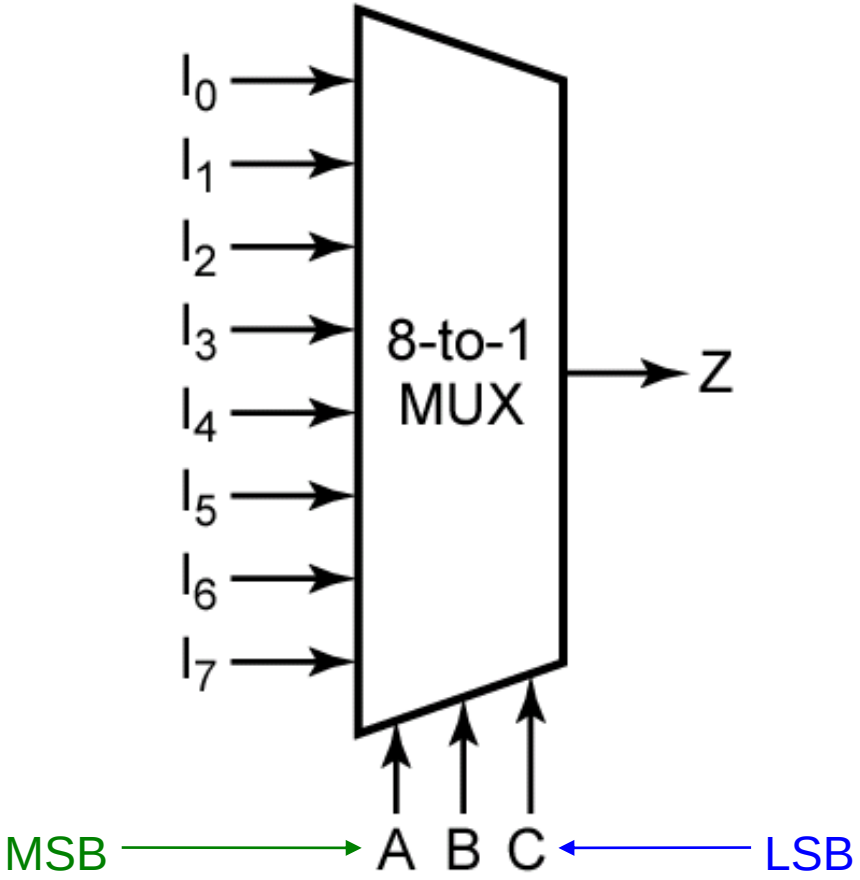
Multiplexers



A	B	F
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

$$Z = A'.B'.I_0 + A'.B.I_1 + A.B'.I_2 + A.B.I_3$$

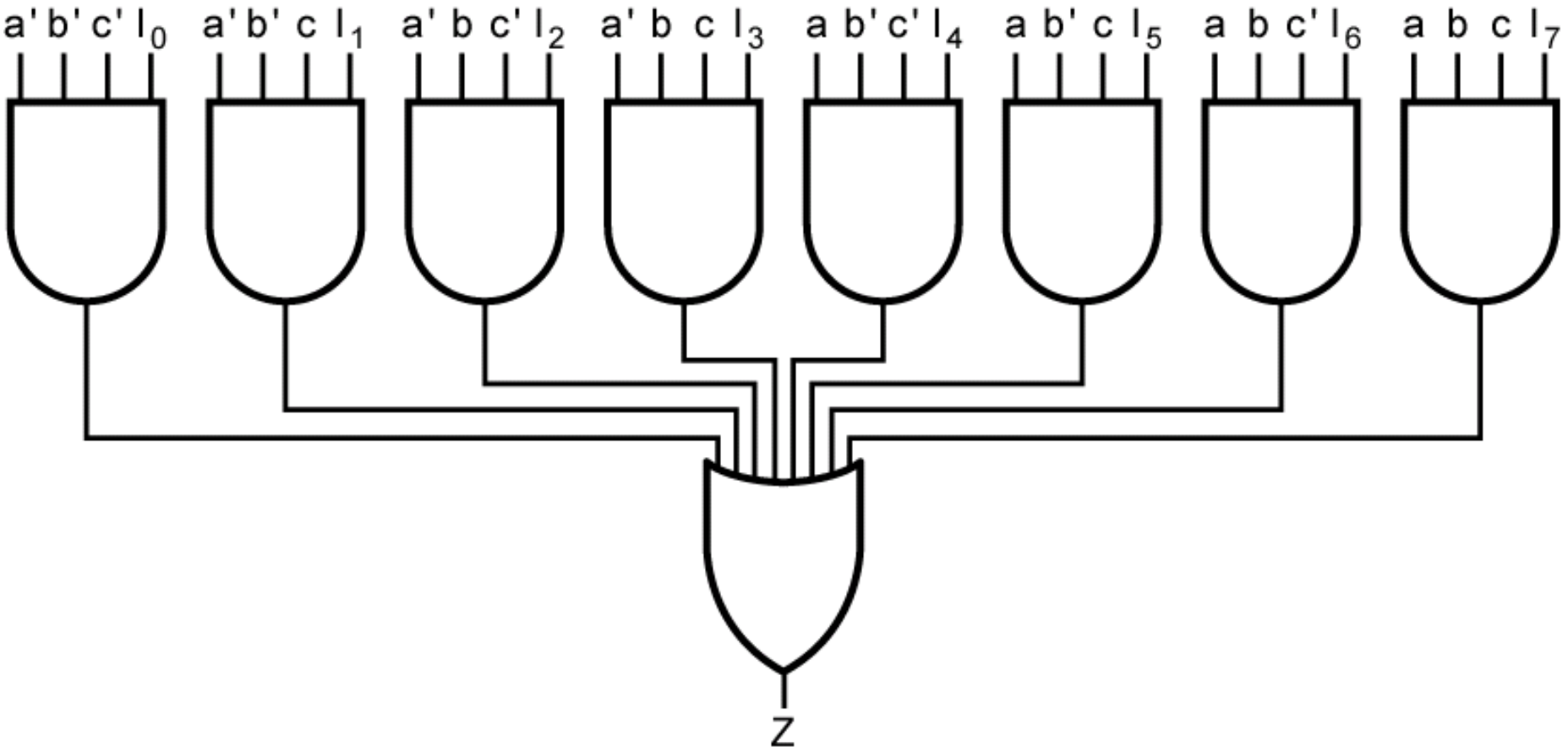
Multiplexers



A	B	C	F
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	I_4
1	0	1	I_5
1	1	0	I_6
1	1	1	I_7

$$Z = A'.B'.C'.I_0 + A'.B'.C.I_1 + A'.B.C'.I_2 + A'.B.C.I_3 + A.B'.C'.I_0 + A.B'.C.I_1 + A'.B.C'.I_2 + A.B.C.I_3$$

Multiplexers



Multiplexers

Exercise:

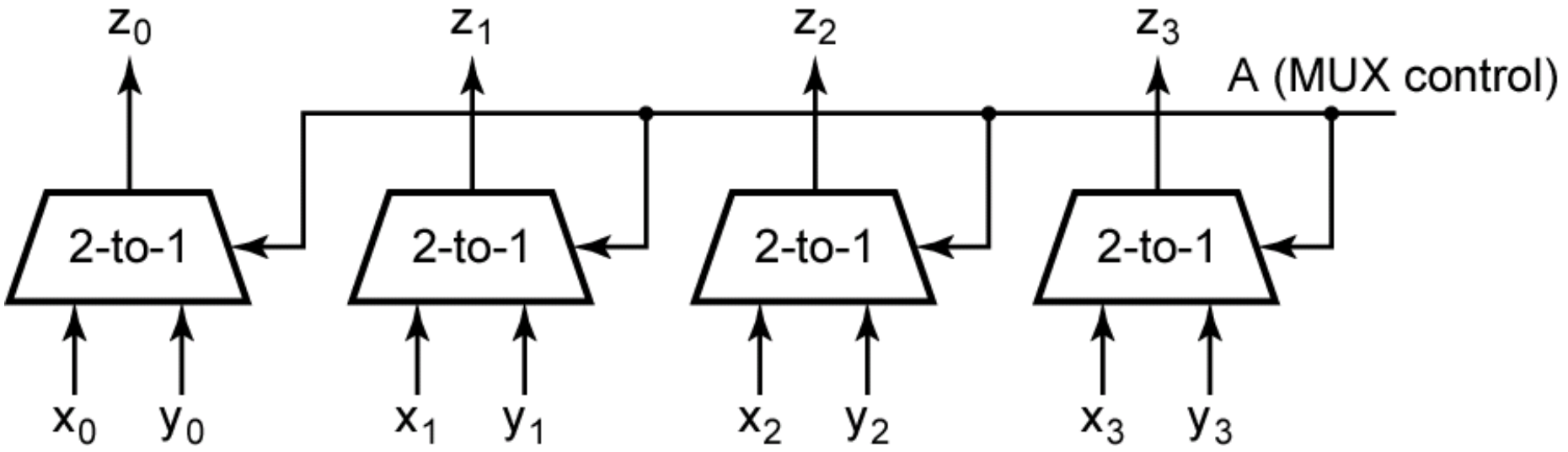
Design an 8-to-1 multiplexer using
4-to-1 and 2-to-1 multiplexers only.

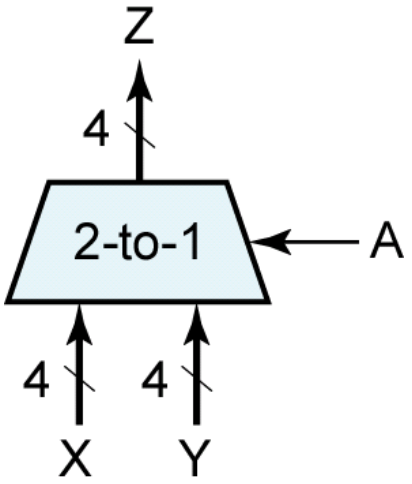
Multiplexers

Exercise:

Design a 16-to-1 multiplexer using
4-to-1 multiplexers only.

Multiplexer (Bus)



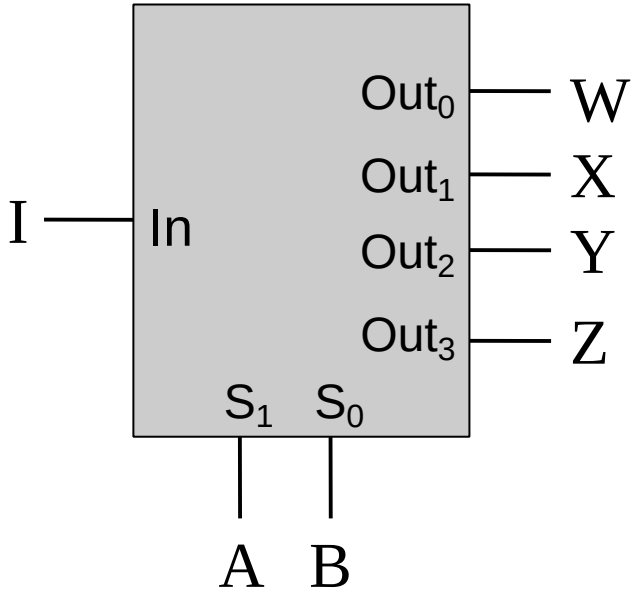


Demultiplexers

Demultiplexers

- A demultiplexer has
  - N control inputs
  - 1 data input
  - 2^N outputs
- A demultiplexer routes (or connects) the data input to the selected output.
  - The value of the control inputs determines the output that is selected.
- A demultiplexer performs the opposite function of a multiplexer.

Demultiplexers



$W = A'.B'.I$

$X = A.B'.I$

$Y = A'.B.I$

$Z = A.B.I$

A	B	W	X	Y	Z
0	0	I	0	0	0
0	1	0	I	0	0
1	0	0	0	I	0
1	1	0	0	0	I

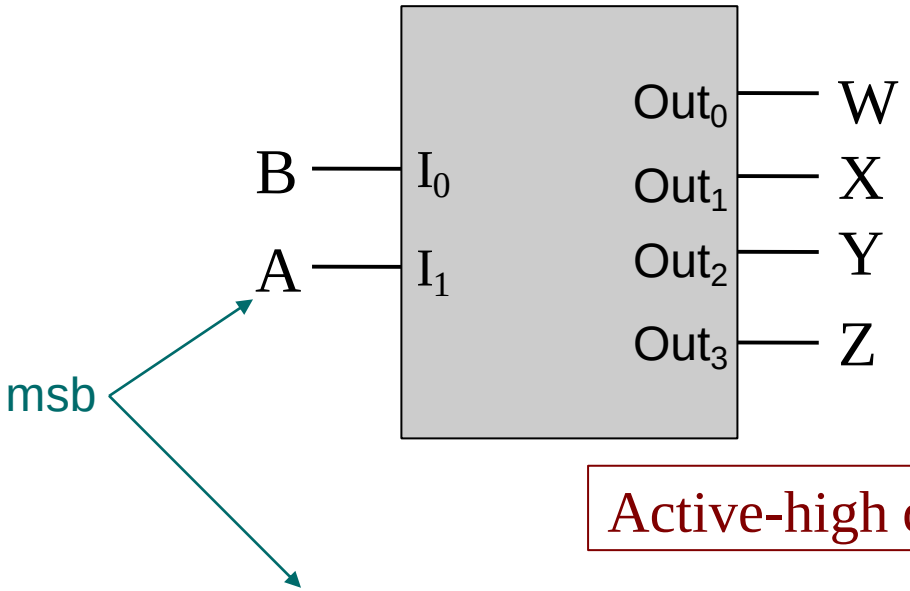
Decoders

Decoders

- A decoder has
  - N inputs
  - 2^N outputs
- A decoder selects one of 2^N outputs by decoding the binary value on the N inputs.
- The decoder generates all of the minterms of the N input variables.
  - Exactly one output will be active for each combination of the inputs.

What does “active” mean?

Decoders



W = A'.B'

X = A.B'

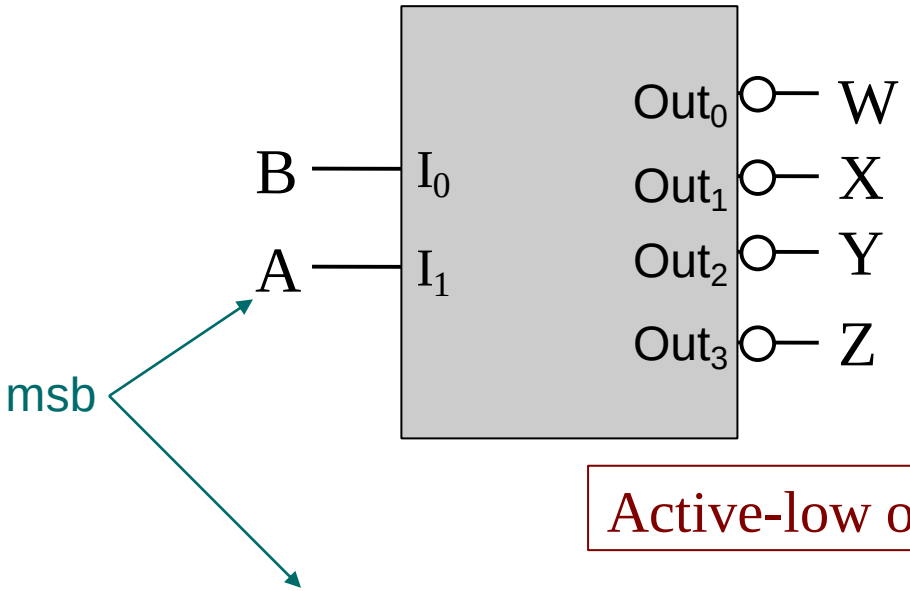
Y = A'.B

Z = A.B

Active-high outputs

A	B	W	X	Y	Z
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

Decoders



W = (A'.B')'

X = (A.B')'

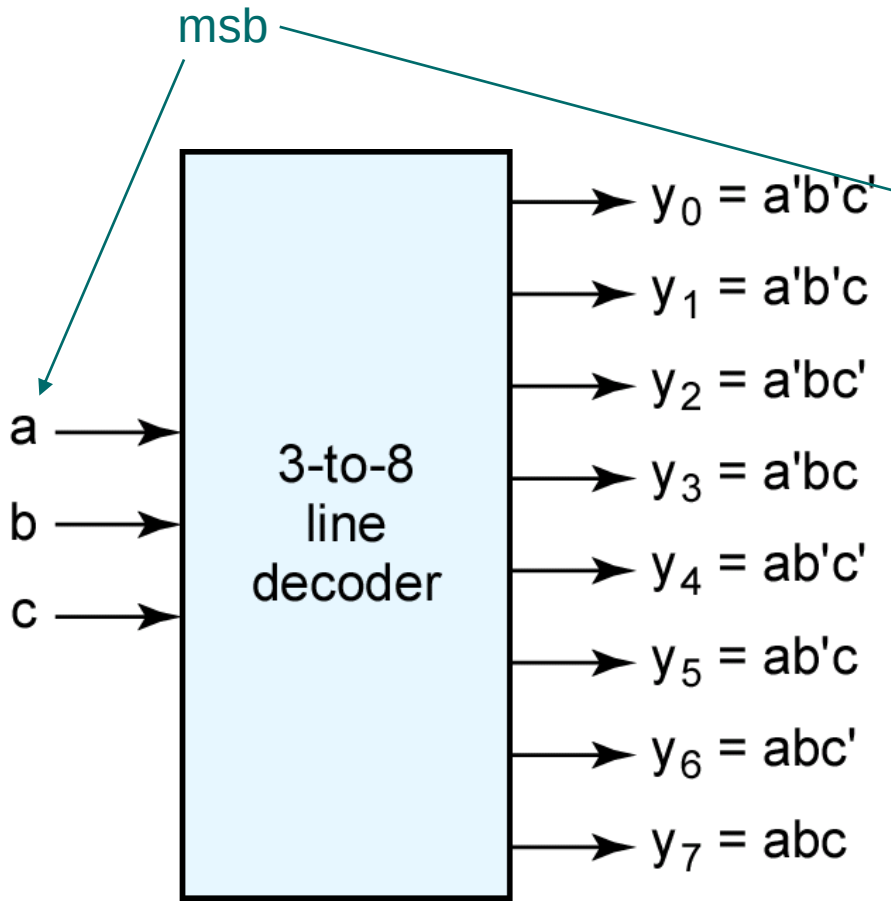
Y = (A'.B)'

Z = (A.B)'

Active-low outputs

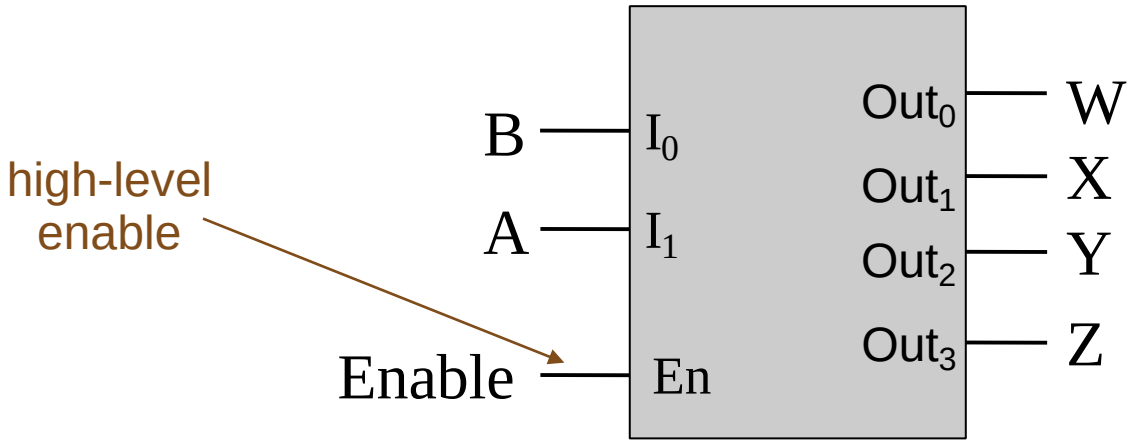
A	B	W	X	Y	Z
0	0	0	1	1	1
0	1	1	0	1	1
1	0	1	1	0	1
1	1	1	1	1	0

Decoders



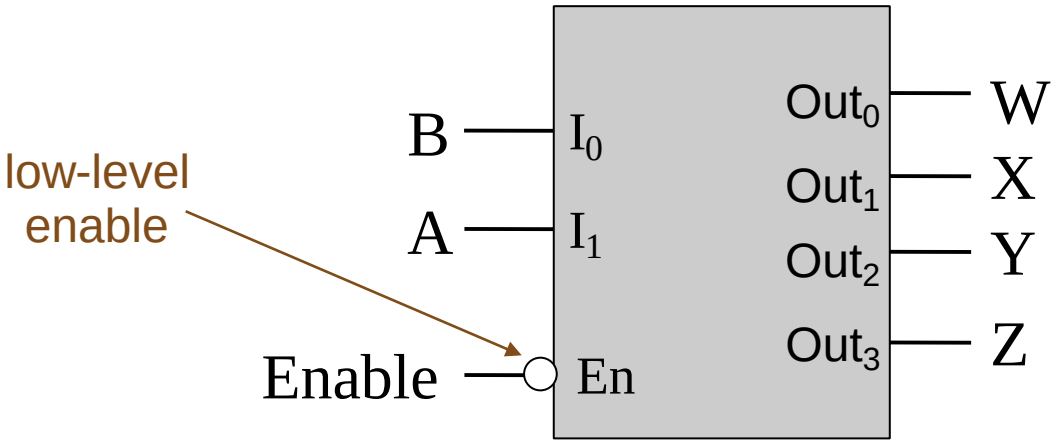
a	b	c	y_0	y_1	y_2	y_3	y_4	y_5	y_6	y_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

Decoder with Enable



	En	A	B	W	X	Y	Z
enabled	1	0	0	1	0	0	0
	1	0	1	0	1	0	0
	1	1	0	0	0	1	0
	1	1	1	0	0	0	1
disabled	0	x	x	0	0	0	0

Decoder with Enable



	En	A	B	W	X	Y	Z
enabled	0	0	0	1	0	0	0
	0	0	1	0	1	0	0
	0	1	0	0	0	1	0
	0	1	1	0	0	0	1
disabled	1	x	x	0	0	0	0

Decoders

Exercise:

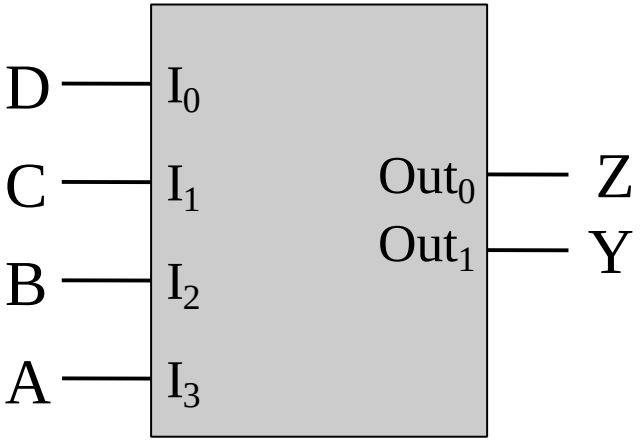
Design a 4-to-16 decoder using
2-to-4 decoders only.

Encoders

Encoders

- An encoder has
  - 2^N inputs
  - N outputs
- An encoder outputs the binary value of the selected (or active) input.
- An encoder performs the inverse operation of a decoder.
- Issues
  - What if more than one input is active?
  - What if no inputs are active?

Encoders



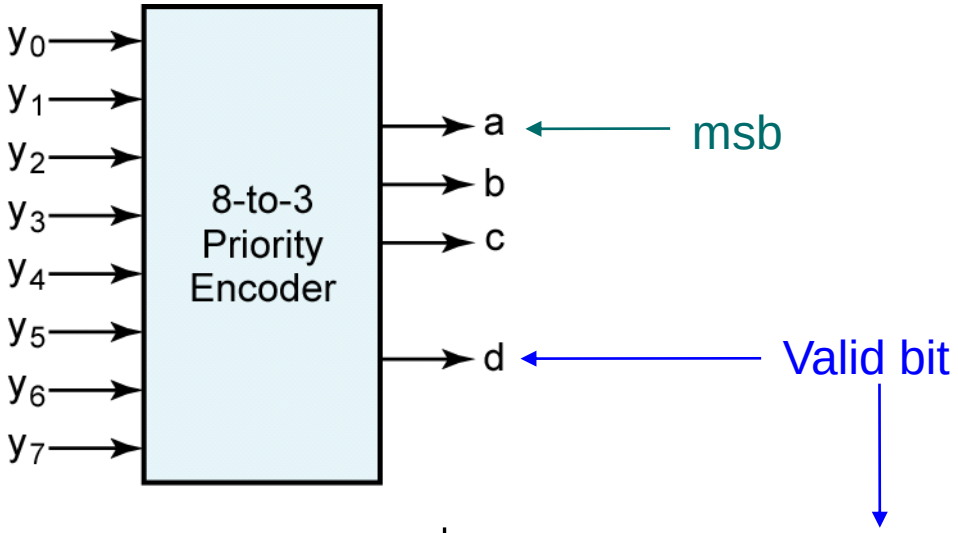
A	B	C	D	**Y**	**Z**
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

Priority Encoders

- If more than one input is active, the higher-order input has priority over the lower-order input.
  - The higher value is encoded on the output
- A valid indicator, d, is included to indicate whether or not the output is valid.
  - Output is invalid when no inputs are active
    - d = 0
  - Output is valid when at least one input is active
    - d = 1

Why is the valid indicator needed?

Priority Encoders



y_0	y_1	y_2	y_3	y_4	y_5	y_6	y_7	a	b	c	d
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	1
X	1	0	0	0	0	0	0	0	0	1	1
X	X	1	0	0	0	0	0	0	1	0	1
X	X	X	1	0	0	0	0	0	1	1	1
X	X	X	X	1	0	0	0	1	0	0	1
X	X	X	X	X	1	0	0	1	0	1	1
X	X	X	X	X	X	1	0	1	1	0	1
X	X	X	X	X	X	X	1	1	1	1	1

Designing logic circuits using multiplexers

Using an *n*-input Multiplexer

- Use an *n*-input multiplexer to realize a logic circuit for a function with *n* minterms.
  - $m = 2^n$, where m = # of variables in the function
- Each minterm of the function can be mapped to an input of the multiplexer.
- For each row in the truth table, for the function, where the output is 1, set the corresponding input of the multiplexer to 1.
  - That is, for each minterm in the minterm expansion of the function, set the corresponding input of the multiplexer to 1.
- Set the remaining inputs of the multiplexer to 0.

Using an *n*-input Mux

Example:

Using an 8-to-1 multiplexer, design a logic circuit to realize the following Boolean function

$$F(A,B,C) = \Sigma m(2, 3, 5, 6, 7)$$

Using an *n*-input Mux

Example:

Using an 8-to-1 multiplexer, design a logic circuit to realize the following Boolean function

$$F(A,B,C) = \Sigma m(1, 2, 4)$$

Using an *(n / 2)*-input Multiplexer

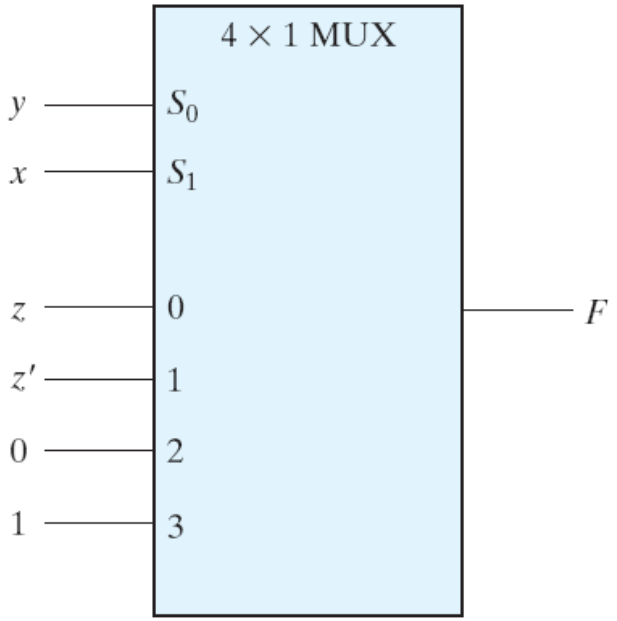
- Use an *(n / 2)*-input multiplexer to realize a logic circuit for a function with *n* minterms.
  - m = 2^n, where m = # of variables in the function
- Group the rows of the truth table, for the function, into *(n / 2)* pairs of rows.
  - Each pair of rows represents a product term of *(m – 1)* variables.
  - Each pair of rows can be mapped to a multiplexer input.
- Determine the logical function of each pair of rows in terms of the m^{th} variable.
  - If the m^{th} variable, for example, is x, then the possible values are x, x', 0, and 1.

Using an *(n / 2)*-input Mux

Example: F(x,y,z) = Σm(1, 2, 6, 7)

x	y	z	F	
0	0	0	0	$F = z$
0	0	1	1	
0	1	0	1	$F = z'$
0	1	1	0	
1	0	0	0	$F = 0$
1	0	1	0	
1	1	0	1	$F = 1$
1	1	1	1	

(a) Truth table

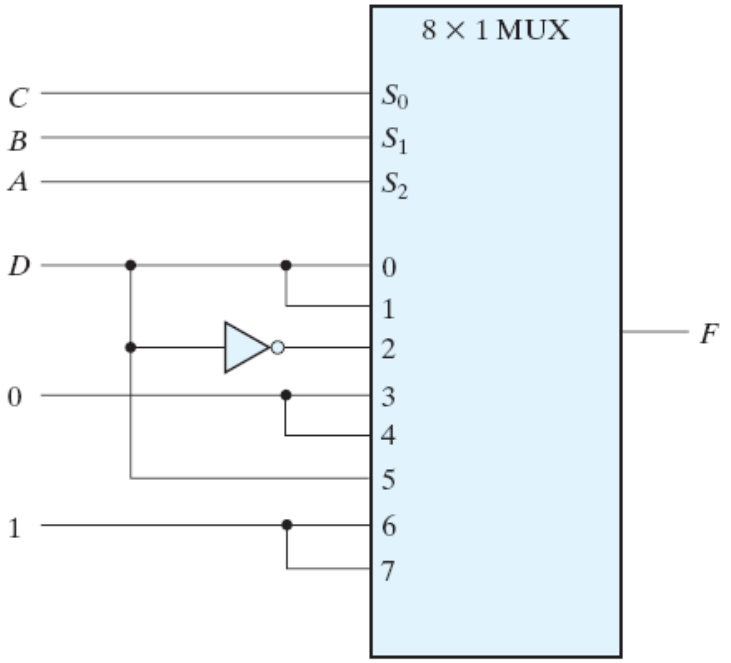


(b) Multiplexer implementation

Using an *(n / 2)*-input Mux

Example: F(A,B,C,D) = Σm(1,3,4,11,12–15)

A	B	C	D	F	
0	0	0	0	0	$F = D$
0	0	0	1	1	
0	0	1	0	0	$F = D$
0	0	1	1	1	
0	1	0	0	1	$F = D'$
0	1	0	1	0	
0	1	1	0	0	$F = 0$
0	1	1	1	0	
1	0	0	0	0	$F = 0$
1	0	0	1	0	
1	0	1	0	0	$F = D$
1	0	1	1	1	
1	1	0	0	1	$F = 1$
1	1	0	1	1	
1	1	1	0	1	$F = 1$
1	1	1	1	1	



Using an *(n / 4)*-input Mux

The design of a logic circuit using an *(n / 2)*-input multiplexer can be easily extended to the use of an *(n / 4)*-input multiplexer.

Designing logic circuits using decoders

Using an *n*-output Decoder

- Use an *n*-output decoder to realize a logic circuit for a function with *n* minterms.
- Each minterm of the function can be mapped to an output of the decoder.
- For each row in the truth table, for the function, where the output is 1, sum (or “OR”) the corresponding outputs of the decoder.
  - That is, for each minterm in the minterm expansion of the function, OR the corresponding outputs of the decoder.
- Leave remaining outputs of the decoder unconnected.

Using an *n*-output Decoder

Example:

Using a 3-to-8 decoder, design a logic circuit to realize the following Boolean function

$$F(A,B,C) = \Sigma m(2, 3, 5, 6, 7)$$

Using an *n*-output Decoder

Example:

Using two 2-to-4 decoders, design a logic circuit to realize the following Boolean function

$$F(A,B,C) = \Sigma m(0, 1, 4, 6, 7)$$

Questions?